Как сделать smooth-motion рилс с iPad

Oct 18, 2026 · @Maksat

Как это устроено

Ролик — это веб-страница. Headless Chrome рисует её кадр за кадром, вызывая `render(t)`, а ffmpeg склеивает кадры со звуком. Монтажной программы нет, поэтому каждый рендер одинаков и пересобирается одной командой. На выходе: 1080×1920, 30 fps, H.264 + AAC, голос с SFX на −14 LUFS. Рилс «Ғафур · Рахим» (84 с) сделан именно так.



пайплайн рилса · 9 шагов, тайм-лайн в центре

Озвучка идёт первой. Из её таймингов получаются метки, и по этим меткам рисуется картинка и раскладываются звуки. Поэтому движение совпадает с речью.

Подготовка на iPad

Работать будешь в Claude Code on the web (claude.ai/code): сессия идёт на облачной машине Ubuntu 24.04 (примерно 4 vCPU и 16 ГБ), iPad только управляет ей. Нужны тариф Claude Pro, Max, Team или Enterprise (на Free облачных сессий нет), аккаунт GitHub и ключ ElevenLabs на тарифе Starter. Окружение личное для каждого аккаунта, поэтому на втором аккаунте всё ниже делается заново.

1. **Репозиторий.** На github.com создай приватный репозиторий с README. Для приватного репозитория нужно поставить Claude GitHub App: claude.ai/code предложит это при подключении.
2. **Стартовые файлы — через веб-интерфейс GitHub** (Add file → Upload files, до 25 МиБ на файл). Загрузи:
   - `CLAUDE.md` — этот документ, экспортированный в Markdown;
   - скриншоты референса в `ref/`. Прикрепление картинок к сообщению в облачной сессии в документации не описано, а файлы в репозитории Claude точно увидит;
   - шрифты в `src/web/fonts/`: KFGQPC Uthmanic Hafs (Комплекс короля Фахда), Scheherazade New (SIL OFL) и открытый латинский/кириллический шрифт вроде Inter. Google Sans из приложения — не открытый шрифт;
   - `src/data/quran.db` из репозитория Qamar (1,7 МБ), если факты считаем по тексту приложения.
3. **Сессия.** Открой claude.ai/code в браузере iPad под вторым аккаунтом, подключи GitHub и выбери репозиторий. Приложение Claude для iPad подойдёт, чтобы следить за сессией и отвечать. Окружение настраивается только на claude.ai/code.
4. **Окружение** (селектор окружения под полем сообщения → новое окружение):
   - **Network access: Custom.** Поставь галку «Also include default list of common package managers» (npm, PyPI, GitHub, apt, `*.googleapis.com`) и добавь по одному домену на строку: `api.elevenlabs.io`, `qamar.web.app`, `cdn.playwright.dev`, `playwright.download.prss.microsoft.com` (браузеры Playwright качаются не с npm). Если текст Корана берёшь с Tanzil, добавь ещё `tanzil.net`. Не сработало — временно включи Full.
   - **Environment variables** (формат `.env`): `ELEVENLABS_API_KEY=…` и `BASH_DEFAULT_TIMEOUT_MS=600000`. Вторая строка разрешает команде идти до 10 минут (по умолчанию — 2). Значения видит каждый, кто пользуется окружением. Надёжнее — network secret: Name, Allowed websites `api.elevenlabs.io`, заголовок `xi-api-key` без префикса. Тогда ключ не виден даже Claude, но с ElevenLabs этот вариант мы не проверяли.
   - **Setup script** — всё, что ставится в саму машину. Версия Playwright здесь и в `package.json` должна совпадать, иначе будет «Executable doesn't exist». Скрипт короче пяти минут кэшируется; ошибка в нём не даст запустить сессию.

```
#!/bin/bash
set -euo pipefail
apt-get update -qq && apt-get install -y -qq ffmpeg
pip install -q numpy pillow imageio-ffmpeg
npm i -g playwright@1.49.1
playwright install --with-deps chromium
```

5. **Долгие команды.** Рендер проси запускать в фоне: фоновая команда может идти до 30 минут сверх лимита. Машина засыпает после нескольких минут без активности, а фоновая работа после этого не восстанавливается. Пока идёт рендер, держи сессию открытой и активной. Надёжнее рендерить кусками по 20–30 с и склеивать их через `ffmpeg -f concat`.
6. **Как забрать mp4.** Сессия пушит ветку, а файл скачиваешь с GitHub на iPad (страница файла → Download). Скачивание файлов прямо из сессии в документации не описано. GitHub не принимает файлы больше 100 МиБ, а каждый перерендер навсегда остаётся в истории. Поэтому черновики держи в `.gitignore`, а коммить только финальный файл.

Структура репозитория

Один рилс — одна папка. Всё, что видно на экране, рисует один файл `web/index.html`. Все времена берутся из `web/timeline.json`, который строится по словам диктора.

```
reels/<имя-рилса>/
  README.md            факты с источниками, сценарий, как пересобрать
  caption.txt          подпись для Instagram
  <имя>.mp4, cover.jpg результат и обложка
  ref/                 скриншоты референса (Claude в облаке Instagram не
откроет)
  src/
    data/              quran.db / перевод — источник фактов
    tools/
      facts.py         считает и проверяет цифры (assert), пишет
web/data.json
      cdp.mjs          headless Chrome «как телефон»: nav, click, type, shot
      sprites.py       вырезает слово из скриншота и перекрашивает
      sheet.py         контакт-лист кадров для проверки
    vo.py              один дубль TTS с таймингами → vo/beats.json
    vo_build.py        биты подряд + метки на словах → web/timeline.json,
vo/track.wav
    sfx.py             синтез звуков + голос сверху → mix.wav (−14 LUFS)
    render.mjs         render(t) для каждого кадра → JPEG → ffmpeg → mp4
```

```
    raw/                реальные скриншоты приложения (1179×2556)
    web/
      index.html        вся сцена: window.render(t), window.READY,
window.TLDUR
      fonts/  shots/    шрифты и подготовленные картинки
    stills/             кадры для проверки (в .gitignore)
```

В корень репозитория положи `CLAUDE.md` — этот документ, экспортированный в Markdown. Тогда каждая новая сессия Claude сразу знает пайплайн и правила.

Шаг за шагом

Один рилс на 80 с — это три-четыре часа работы с Claude. Больше всего времени уходит на шаги 2, 7 и 9.

1. **Референс.** Сделай на iPad 8–12 скриншотов рилса по ходу сюжета и скопируй подпись под ним. Приложи их к первому сообщению или закоммить в `ref/`. Claude в облаке не зайдёт в Instagram, а скачивать чужое видео не нужно.
2. **Факты.** Каждую цифру из референса пересчитай сам скриптом `tools/facts.py` с `assert` на каждое число.
   - Источник текста: `quran.db` из репозитория Qamar (`composeApp/src/wasmJsMain/resources/quran.db`, таблицы `verses` и `quran_paged`) или текст Uthmani с tanzil.net.
   - Счёт зависит от того, как резать слова. Убирай харакаты, артикль ٱل и приставки و ف ب ل ك, но сверяй список случаев глазами.
   - Ищи контрпримеры к обобщениям. В нашем рилсе «парные имена не меняются» оказалось неправдой: عليم حكيم меняется 7 раз.
   - Тафсир и цитаты учёных — только со страницы, которую Claude реально открыл. Если мнений несколько, напиши это на экране.
3. **Сценарий.** Распиши его на 10–13 битов по одной мысли в списке `BEATS` в `vo.py`. Числа пиши словами («жетпіс екі»), бренд — на языке озвучки («Қамар»). Перевод аята бери дословно из приложения, без скобок переводчика. 900–1000 символов дают около 70 с речи.
4. **Озвучка.** Закажи один непрерывный дубль ElevenLabs `/with-timestamps` на весь сценарий (код — в разделе «Ключевой код»). Режь его на биты по тишине между фразами, ускорь `atempo 1.12` и сохрани время каждого слова в `vo/beats.json`. Дубль кэшируй по хэшу текста, чтобы повторный запуск не тратил кредиты.

5. **Тайм-лайн.** `vo_build.py` ставит биты подряд с зазорами `EXTRA` (сколько секунд картинке нужно до бита) и считает метки по словам: `m['flip'] = word('once', 'керісінше')`. Он же печатает `← SHORT` там, где между сценами меньше секунды, — там увеличивай `EXTRA`.
6. **Экраны приложения.** Открой qamar.web.app в headless Chrome с чистым профилем, 393×852 @3x, с локалью языка рилса. Зайди гостем, выставь тему и акцент и сними нужные экраны в `raw/` покадрово (тап → снимок), чтобы потом анимировать. Слово в Мусхафе подсвечивай его собственными пикселями, а не цветным прямоугольником: см. «Подводные камни».
7. **Сцена.** Напиши `web/index.html` с одной функцией `window.render(t)`, которая рисует кадр для любого `t`. Правила движения — в разделе «Секрет плавности».
8. **Звук.** `sfx.py` синтезирует на numpy тапы, whoosh, щипки (Karplus–Strong), колокольчики и дождь на те же метки из `timeline.json`. Подложка идёт на 9 дБ тише голоса и дакается под речью ещё на 8 дБ. Итог нормализуется до −14 LUFS через `ffmpeg -af ebur128`. Музыки нет.
9. **Проверка кадров.** `node render.mjs --stills 3,12.9,24,40` сохраняет отдельные кадры, `tools/sheet.py` склеивает их в контакт-лист. Пусть Claude сам смотрит листы и чинит наложения, пустоты и переносы, а тебе показывает 2–3 листа. Отдельно проверь, сколько секунд держится главный кадр: у нас он сначала держался 0,4 с.
10. **Рендер.** `node render.mjs ../<имя>.mp4 30`: 1080×1920, H.264 `-crf 16 -preset slow`, AAC 192k, `+faststart`. На Mac 84 с ролика рендерятся за 2¼ минуты. В облаке закладывай в несколько раз больше — это оценка, не замер.
11. **Публикация.** Обложка — самый сильный кадр (`ffmpeg -ss 12.9 -frames:v 1 cover.jpg`). Подпись должна быть короче 2200 символов, с фактами, источником перевода и призывом. Закоммить всё и запушь, затем скачай mp4 на iPad с GitHub (предел GitHub — 100 МБ на файл, наш весит 20 МБ).

Секрет плавности

Плавность даёт не монтажная программа, а то, что каждый кадр — чистая функция времени `render(t)`. Браузер ничего не «проигрывает»: для кадра 1234 он рисует ровно `t = 1234 / 30`, и только потом делается снимок. Поэтому нет рывков, пропущенных кадров и рассинхрона со звуком.

- **Никаких CSS-анимаций,** `transition`**,** `setTimeout` **и** `requestAnimationFrame`**.** Они идут по часам браузера, а не по номеру кадра. Всё считается из `t`.

- **Меняй только `opacity` и `transform`** (translate / scale) через один хелпер `S(el, прозрачность, x, y, масштаб)`. Текст при этом остаётся векторным и чётким даже при увеличении ×3.
- **Каждое движение идёт по easing, а не линейно.** `eo` (ease-out cubic) — для появления, `eio` (in-out) — для перелётов, `eback` — для лёгкого «пружинистого» акцента на цифрах.
- **Появление — это прозрачность плюс небольшой подъём на 12–24 px** за 0,4–0,5 с (`up(t, a)`). Исчезновение — 0,3 с. Сцена перекрывается со следующей (`win(t, старт, конец, 0.35, 0.3)`), жёстких склеек нет.
- **Стаггер.** Группа элементов заходит волной с сдвигом 10–70 мс на элемент. Так 72 плитки появляются каскадом и так же золотятся.
- **Один объект перетекает в следующий кадр, а не исчезает.** Плитка 34:2 вырастает из сетки в центр (×3,3). Строка Мусхафа поднимается из телефона линзой. Экраны приложения сменяются горизонтальным push. Глаз следит за одной точкой.
- **Фон живой, но тихий.** Медленный дрейф (−10 px за 10 с), мерцающие звёзды, дождь и искры. Для частиц используй сидовый PRNG (mulberry32), не `Math.random()`: кадр должен выглядеть одинаково при любом перерендере.
- **Картинка привязана к словам диктора.** Цифра бежит на «қырық», дождь усиливается на «төмен», тап падает на «Мұсхафынан». Когда движение совпадает с речью, ролик воспринимается как одно дыхание.
- **Главному кадру — время постоять.** Не меньше 1–1,5 с в полном размере перед следующей сценой. Плавность — это ещё и паузы.
- **Кадровая частота.** 30 fps достаточно, когда всё идёт по easing. Для быстрых перелётов можно рендерить 60 fps (вдвое дольше) и либо так и отдать, либо свести в 30 с лёгким размытием движения: `-vf "tmix=frames=2,fps=30"`.
- **Шрифты и картинки загружаются до первого кадра.** Страница ставит `window.READY = true` только после `document.fonts.load(...)` и `img.decode()`. Иначе в первых кадрах мелькнет запасной шрифт.
- **Одна тема, два смысловых цвета.** Цвета бери из токенов приложения, тогда реальные скриншоты и нарисованные сцены выглядят одним фильмом. В нашем рилсе фон #0D0D12, синий #7EB6FF для Рахим и золото #D9B36B для Ғафур.

Ключевой код

Три куска нельзя делать по-другому: рендер по кадрам, каркас `render(t)` и один дубль озвучки с таймингами. Остальное Claude напишет под конкретный рилс.

render.mjs — кадр за кадром, Linux и Playwright

Система (ffmpeg, Chromium, numpy) ставится setup-скриптом из раздела «Подготовка на iPad». В самом репозитории достаточно `npm i -E playwright@1.49.1` — та же версия, что и там.

```
// node render.mjs ../reel.mp4 30        весь фильм (нужен mix.wav)
// node render.mjs --stills 3,12.9,40    кадры для проверки → stills/
import { chromium } from 'playwright';
import { spawn, execSync } from 'node:child_process';
import { createServer } from 'node:http';
import { readFile, writeFile, mkdir } from 'node:fs/promises';
import { extname, join } from 'node:path';

const args = process.argv.slice(2);
const STILLS = args[0] === '--stills' ? args[1].split(',').map(Number) :
null;
const OUT = STILLS ? null : (args[0] || 'out.mp4'), FPS = Number((!STILLS &&
args[1]) || 30);
const FFMPEG = execSync('python3 -c "import imageio_ffmpeg;
print(imageio_ffmpeg.get_ffmpeg_exe())"').toString().trim();
const TYPES = { '.html': 'text/html', '.json': 'application/json', '.png':
'image/png', '.jpg': 'image/jpeg',
  '.woff2': 'font/woff2', '.ttf': 'font/ttf', '.js': 'text/javascript' };
const server = createServer(async (req, res) => {
  try {
    const p = join('web', decodeURIComponent(new URL(req.url,
'http://x').pathname));
    res.writeHead(200, { 'content-type': TYPES[extname(p)] ||
'application/octet-stream' }); res.end(await readFile(p));
  } catch { res.writeHead(404); res.end(); }
}).listen(8790);

const browser = await chromium.launch();
const page = await browser.newPage({ viewport: { width: 1080, height: 1920 },
deviceScaleFactor: 1 });
await page.goto('http://127.0.0.1:8790/index.html');
await page.waitForFunction(() => window.READY === true, null, { timeout:
30000 });
const cdp = await page.context().newCDPSession(page);          // CDP-снимок
```

```
быстрее page.screenshot
const shot = async (t, q) => {
  await page.evaluate(t => window.render(t), t);
  const { data } = await cdp.send('Page.captureScreenshot', { format: 'jpeg',
quality: q });
  return Buffer.from(data, 'base64');
};

if (STILLS) {
  await mkdir('stills', { recursive: true });
  for (const t of STILLS) await writeFile(`stills/t_${t}.jpg`, await shot(t,
85));
} else {
  const dur = await page.evaluate(() => window.TLDUR), n = Math.round(dur *
FPS);
  const ff = spawn(FFMPEG, ['-v', 'error', '-y', '-f', 'image2pipe', '-
framerate', String(FPS), '-c:v', 'mjpeg', '-i', '-',
    '-i', 'mix.wav', '-map', '0:v', '-map', '1:a',
    '-vf', 'scale=out_color_matrix=bt709:out_range=tv,format=yuv420p',
    '-c:v', 'libx264', '-preset', 'slow', '-crf', '16', '-profile:v', 'high',
    '-colorspace', 'bt709', '-color_primaries', 'bt709', '-color_trc',
'bt709', '-color_range', 'tv',
    '-c:a', 'aac', '-b:a', '192k', '-ar', '48000', '-t', String(dur), '-
movflags', '+faststart', OUT],
    { stdio: ['pipe', 'inherit', 'inherit'] });
  for (let f = 0; f < n; f++) {
    if (!ff.stdin.write(await shot(f / FPS, 95))) await new Promise(r =>
ff.stdin.once('drain', r));
    if (f % 150 === 0) console.log(`frame ${f}/${n}`);
  }
  ff.stdin.end(); await new Promise(r => ff.on('close', r));
}
await browser.close(); server.close();
```

web/index.html — каркас сцены

```
<!doctype html><html><head><meta charset="utf-8"><style>
@font-face{font-family:'KFGQPC';src:url(fonts/qpc_hafs_regular.ttf)}
html,body{margin:0;width:1080px;height:1920px;overflow:hidden;background:#0D0
D12;color:#F3F3F5;font-family:Inter,sans-serif}
.a{position:absolute;opacity:0;will-change:opacity,transform}
.c{left:0;right:0;text-align:center}
.ar{font-family:'KFGQPC';direction:rtl}
</style></head><body>
<canvas id="fx" width="1080" height="1920" style="position:absolute;inset:0">
</canvas>
```

```
<div id="title" class="a c" style="top:200px;font-size:110px;font-
weight:700">Гафур · Рахим</div>
<script>
(async () => {
const $ = id => document.getElementById(id);
const clamp = (x, a = 0, b = 1) => Math.max(a, Math.min(b, x));
const ramp = (t, a, b) => clamp((t - a) / (b - a));
// 0 → 1 между a и b
const eo = u => 1 - (1 - u) ** 3;
// ease-out: появление
const eio = u => u < .5 ? 4 * u ** 3 : 1 - (-2 * u + 2) ** 3 / 2;
// ease-in-out: перелёты
const win = (t, a, b, fi = .35, fo = .3) => clamp((t - a) / fi) * clamp((b -
t) / fo);   // жизнь сцены
const up = (t, a, d = 18) => d * (1 - eo(ramp(t, a, a + .5)));
// подъём при появлении
function S(el, o, x = 0, y = 0, s = 1) {
  el = typeof el === 'string' ? $(el) : el;
  el.style.opacity = o.toFixed(3);
  el.style.transform = `translate(${x.toFixed(1)}px,${y.toFixed(1)}px)
scale(${s.toFixed(4)})`;
}
const TL = await (await fetch('timeline.json')).json(), M = TL.marks;
window.TLDUR = TL.duration;
await document.fonts.load('60px KFGQPC'); await document.fonts.ready;
await Promise.all([...document.images].map(i => i.decode().catch(() => {})));

window.render = t => {                        // только t и M — никакого
состояния между кадрами
  const o1 = win(t, 0, M.ruleIn);
  S('title', o1 * ramp(t, M.g1 - .25, M.g1 + .1), 0, up(t, M.g1 - .25, 24));
  // … каждый элемент: S(id, прозрачность, x, y, масштаб); частицы — на #fx
из сидового PRNG
};
window.render(0); window.READY = true;
})();
</script></body></html>
```

vo.py — один дубль с таймингами

```
script = " ".join(text for _, text in BEATS) + " …"   # v3 глотает последнее
слово без хвоста
body = {"text": script, "model_id": MODEL, "language_code": "kk",
        "voice_settings": {"stability": 0.5, "similarity_boost": 0.75}}
req = urllib.request.Request(
    f"https://api.elevenlabs.io/v1/text-to-speech/{VOICE}/with-timestamps?
output_format=mp3_44100_128",
    data=json.dumps(body).encode(),
    headers={"xi-api-key": os.environ["ELEVENLABS_API_KEY"], "Content-Type":
"application/json"})
r = json.load(urllib.request.urlopen(req, timeout=300))
mp3 = base64.b64decode(r["audio_base64"])
al = r["alignment"]   # characters + character_start_times_seconds +
character_end_times_seconds
# дальше: найти каждый бит в "".join(al["characters"]), резать посередине
тишины между битами,
# atempo 1.12 через ffmpeg, времена слов делить на 1.12 → vo/beats.json
```

Язык	Голос	voice_id	Модель
Қазақша	Nurlan	`dQO3cohklRk58SrkwMjq`	`eleven_v3_conversational`
Русский	Stanislav	`ogi2DyUAKJb7CEdqqvlU`	`eleven_v3`
English	George	`JBFqnCBsd6RMkjVDRZzb`	`eleven_multilingual_v2`

Голоса из библиотеки через API и коммерческая лицензия доступны с тарифа ElevenLabs Starter. На бесплатном тарифе будет 402. Starter отдаёт только `mp3_44100_128`.

Скриншоты приложения в облаке

Claude на iPad не может снять экран твоего iPad или iPhone. Сессия идёт на облачной Linux-машине Anthropic, а iPad только показывает чат. К твоим устройствам эта машина не подключена, а iPadOS не даёт ни одному приложению снимать чужой экран. Экраны Qamar для ролика можно получить тремя способами.

Способ	Что попадёт в кадр	Что нужно
Claude снимает qamar.web.app в облаке (основной)	веб-версия, любая тема, акцент и язык, чистый гостевой профиль	домен `qamar.web.app` в Custom-сети, Chromium из setup-скрипта, `tools/capture.mjs` ниже
Снимаешь сам на iPhone	нативное приложение так, как его видит пользователь	боковая кнопка + громкость вверх, затем PNG в `src/raw/` через GitHub (Add file → Upload files)
Remote Control с Mac	всё, что доступно на компьютере, как в сессии на Mac	включённый Mac, на нём Claude Code в том же аккаунте; iPad только управляет (описание)

Свои скриншоты делай в режиме «Не беспокоить» и с полной батареей, иначе в кадр попадут уведомления. Размер зависит от модели: iPhone 14 Pro, 15, 15 Pro и 16 дают 1179×2556, как наши захваты, а 16 Pro — 1206×2622. Для других моделей проси Claude масштабировать.

tools/capture.mjs — «айфон» в headless Chromium

Браузер запускается один раз и живёт между командами, поэтому Claude идёт по приложению по шагам: тап, снимок, посмотрел, следующий тап. Профиль временный, в свой аккаунт здесь не входи. Нужен Node 22 (встроенный `WebSocket`) и Playwright той же версии, что в setup-скрипте.

```
// node tools/capture.mjs start                    браузер с чистым профилем
(LOCALE=kk-KZ по умолчанию)
// node tools/capture.mjs nav https://qamar.web.app [ждать мс]
// node tools/capture.mjs click <x> <y> [ждать мс]    CSS px в кадре 393×852
// node tools/capture.mjs type <текст>
// node tools/capture.mjs scroll <x> <y> <dy>
// node tools/capture.mjs shot src/raw/<имя>.png    снимок 1179×2556
// node tools/capture.mjs stop
import { spawn } from 'node:child_process';
import { writeFileSync, mkdtempSync } from 'node:fs';
import { tmpdir } from 'node:os';
import { join } from 'node:path';
import { chromium } from 'playwright';

const PORT = +(process.env.CDP_PORT || 9222), W = 393, H = 852, DPR = 3,
```

```
LOCALE = process.env.LOCALE || 'kk-KZ';
const [cmd, ...args] = process.argv.slice(2);
const sleep = ms => new Promise(r => setTimeout(r, ms));

if (cmd === 'start') {
  spawn(chromium.executablePath(), ['--headless=new', `--remote-debugging-
port=${PORT}`,
    `--user-data-dir=${mkdtempSync(join(tmpdir(), 'qamar-'))}`, '--no-
sandbox', '--disable-dev-shm-usage',
    '--use-angle=swiftshader', '--enable-unsafe-swiftshader', '--hide-
scrollbars',
    `--lang=${LOCALE}`, `--window-size=${W},${H}`, 'about:blank'], { stdio:
'ignore', detached: true }).unref();
  await sleep(2000); console.log('started'); process.exit(0);
}

const target = (await (await
fetch(`http://127.0.0.1:${PORT}/json`)).json()).find(t => t.type === 'page');
const ws = new WebSocket(target.webSocketDebuggerUrl);
await new Promise(r => ws.addEventListener('open', r));
let id = 0; const pending = new Map();
ws.addEventListener('message', e => { const m = JSON.parse(e.data); if (m.id
&& pending.has(m.id)) { pending.get(m.id)(m); pending.delete(m.id); } });
const send = (method, params = {}) => new Promise(r => { const i = ++id;
pending.set(i, r); ws.send(JSON.stringify({ id: i, method, params })); });
// переопределения живут, пока подключена команда, поэтому ставим их каждый
раз;
// --window-size держит тот же размер между командами, и вёрстка не прыгает
await send('Emulation.setDeviceMetricsOverride', { width: W, height: H,
deviceScaleFactor: DPR, mobile: true });
await send('Emulation.setLocaleOverride', { locale: LOCALE });
const mouse = (type, x, y) => send('Input.dispatchMouseEvent', { type, x: +x,
y: +y, button: 'left', clickCount: 1 });
switch (cmd) {
  case 'nav': await send('Page.navigate', { url: args[0] }); await sleep(+
(args[1] || 10000)); break;
  case 'click': await mouse('mouseMoved', args[0], args[1]); await
mouse('mousePressed', args[0], args[1]);
    await sleep(60); await mouse('mouseReleased', args[0], args[1]); await
sleep(+(args[2] || 1500)); break;
  case 'type': await send('Input.insertText', { text: args[0] }); await
sleep(800); break;
  case 'scroll': await send('Input.dispatchMouseEvent', { type: 'mouseWheel',
x: +args[0], y: +args[1], deltaX: 0, deltaY: +args[2] });
    await sleep(1200); break;
  case 'shot': { const s = await send('Page.captureScreenshot', { format:
```

```
'png' });
    writeFileSync(args[0], Buffer.from(s.result.data, 'base64'));
console.log('wrote', args[0]); break; }
  case 'stop': await send('Browser.close'); break;
}
ws.close(); process.exit(0);
```

Как Claude идёт по приложению

1. После каждого `shot` уменьшает снимок в 3 раза (393×852) и смотрит его. Тогда координаты на картинке совпадают с координатами для `click`.
2. Гостевой вход на казахском: «Өткізіп жіберу» → «Тіркелгісіз жалғастыру» → Almaty → «Орнатуды аяқтау» → «Қазір емес».
3. Тема и акцент: меню → «Баптаулар» → «Тақырып: Қолмен» → нужная тема → «Акцент түсі».
4. Мусхаф: поиск «34:2» → «Бет режимі» → настройки чтения: «Мусхаф», «Тәжуид түстері» выключить. Тап по аяту → «Аударма» → «Жүктеу»: перевод скачивается с `firebasestorage.googleapis.com`, он входит в список `*.googleapis.com` по умолчанию.
5. Всё, что будет анимировано (счётчик, выделение, панель перевода), снимается по снимку на каждое состояние: `tasbih21.png`, `tasbih22.png` и так далее.
6. Вместо приложения пустой экран — значит, либо домен не разрешён, либо не поднялся WebGL. Пусть Claude посмотрит консоль страницы через `Runtime.evaluate`. Флаги SwiftShader в `start` нужны именно для этого, но на облачной машине мы их не проверяли.

Если Claude на iPad снова скажет, что не может сделать скриншоты, напиши ему: «Моё устройство не нужно. Запусти tools/capture.mjs, открой qamar.web.app и сними экраны в своей облачной машине по разделу «Скриншоты приложения в облаке» в CLAUDE.md».

Стартовый промпт

Скопируй его в первую сессию и подставь своё в угловых скобках. Главное в нём — две остановки: на сценарии (до того, как потрачены кредиты на озвучку) и на контакт-листах (до полного рендера).

```
Прочитай CLAUDE.md — это наш пайплайн рилсов. Сделай новый рилс в
reels/<имя>/.

Референс: скриншоты в ref/<имя>/ и его подпись в ref/<имя>/caption.txt
(оригинал: <ссылка на Instagram>). Нужно «вот такое видео», но в нашем стиле:
1080×1920, 30 fps, плавное движение по правилам из раздела «Секрет
плавности».
Язык: <қазақша>. Голос: <Nurlan> из ElevenLabs (ключ в ELEVENLABS_API_KEY) +
синтезированные SFX, без музыки.
Тема и акцент: из цветов Qamar, выбери сам и объясни одной строкой почему.
Упомяни функции Qamar, которые по теме: <Мұсхаф с переводом, 99 имён, счётчик
зикра…>.
Обещай только то, что видно в qamar.web.app сейчас, и снимай реальные экраны
оттуда.

Порядок:
1. Перескажи референс по битам и предложи сценарий: 10–13 битов, 900–1000
символов текста,
   что идёт на экран в каждом. Стоп — жди моего ОК.
2. Пересчитай каждую цифру скриптом с assert по src/data/quran.db. Ищи
контрпримеры к любому
   обобщению. Тафсир и цитаты — только со страниц, которые ты открыл. Скажи,
что не сошлось.
3. Озвучка одним дублем с таймингами → биты → тайм-лайн с метками на словах.
4. Экраны приложения: headless Chrome, чистый профиль, 393×852 @3x, гостевой
вход.
5. Сцена web/index.html с render(t). После каждых двух-трёх сцен сам смотри
контакт-листы
   кадров, чини наложения и пустоты и показывай мне лист файлом в stills/.
6. Звук (−14 LUFS), полный рендер в фоне или кусками, обложка, caption.txt,
README.md с фактами и источниками.
7. Закоммить исходники и один финальный mp4, запушь. В конце напиши, какие
слова в озвучке
   мне нужно переслушать.
```

Для следующих рилсов хватит короткого варианта: «Новый рилс по CLAUDE.md. Референс в ref/<имя>/, язык <…>, скопируй папку прошлого рилса как старт».

Подводные камни и чеклист

Каждый пункт ниже уже сломал один из наших рилсов хотя бы один раз.

Подводные камни

- **Озвучка по строчкам «рвётся».** Отдельные запросы на каждую фразу дают обрезанные начала и хвосты. Нужен один дубль, разрезанный по тишине.
- **eleven_v3 съедает последнее слово.** Добавляй в конец текста « …».
- **Распознавание речи может быть недоступно.** У нашего ключа speech-to-text возвращал 401. Имена и арабские слова переслушивай сам.
- **Арабские слова на скриншоте ищи по пикселям, а не глазами.** На кропах Claude часто путает порядок слов справа налево. Строки находи по профилю чернил, слова — по пробелам и связным компонентам. В шрифтах Мусхафа соседние слова часто соприкасаются.
- **Подсветка слова — его собственные чернила другого цвета.** На светлой странице альфа = насколько пиксель темнее фона, на тёмной — насколько светлее. Цветные метки аятов выкидывай по насыщенности. Чтобы слова читались на телефоне, давай линзу — увеличенную строку.
- **У каждого скриншота своя верхняя панель.** Если слои скроллятся, обрезай панель у каждого, иначе она въедет в кадр посреди движения.
- **У всех абсолютных элементов должен быть задан `top`.** У нас подписи без него прилипли к верху кадра. Длинную арабскую строку в плитке мерь через `scrollWidth` и уменьшай шрифт под ширину.
- **Безопасная зона Instagram.** Сверху и особенно снизу (подпись, кнопки) кадр закрыт интерфейсом. Важный текст держи примерно в полосе у 200–1500 px и подальше от правого края (кнопки лайка). Это наблюдение, а не спецификация.
- **Обобщения референса не бери на веру.** Из популярных «равных пар» слов в Коране большинство на пересчёте не сходится.
- **Не обещай того, чего нет в приложении.** Например, поиск Qamar ищет ссылки вида 34:2, а не арабские слова. В финале ставь «iPhone — qamar.web.app, Android — Google Play»: значка App Store нет, потому что Qamar сейчас вне App Store.
- **Цвета на iPhone «плывут» без тегов bt709.** Они уже вписаны в `render.mjs` выше.
- **Чужое видео и чужая музыка не нужны.** Берём только идею, всё остальное делаем сами.

Чеклист перед публикацией

- [ ] Каждая цифра пересчитана скриптом с assert
- [ ] Перевод аята дословно из приложения, переводчик назван

- [ ] Цитаты учёных взяты со страниц, которые реально открыты; другие мнения упомянуты
- [ ] Озвучка прослушана целиком, имена произнесены верно
- [ ] На контакт-листах нет наложений, пустой кадр не дольше секунды, главный кадр держится больше секунды
- [ ] Важный текст внутри безопасной зоны
- [ ] Показанные функции есть в qamar.web.app сейчас, значка App Store нет
- [ ] Громкость −14 LUFS, пик ниже −1 dBFS
- [ ] Обложка выбрана, подпись короче 2200 символов
- [ ] Финальный mp4 закоммичен один раз и меньше 100 МиБ

Источники

- Claude Code on the web — быстрый старт (тарифы, GitHub)
- Облачные окружения: сеть, переменные, секреты, setup-скрипт, лимиты
- Claude Code на телефоне и iPad
- Хук SessionStart и CLAUDE_CODE_REMOTE
- Playwright: откуда качаются браузеры
- GitHub: большие файлы и лимит 100 МиБ
- Tanzil — текст Корана для проверки фактов
- Референс: quran_analyzed, «Ghafūr. Raḥīm.»